Université Jean Monnet Saint-Étienne

Master in Machine Learning and Data Mining (MLDM)

Automatic Road Damage Detection from Street Images using Deep Learning

Project Report

Student: Muhammad Sultan Haidar Cheema
Programme: M1 Machine Learning and Data Mining
Academic year: 2025–2026
Supervisor: *Amaury Habrard*

August 20, 2026

Contents

1 Introduction **1**

1.1 Background and Motivation 1
1.2 Project Context 1
1.3 Problem Statement 1
1.4 Objectives 1
1.5 Project Workflow 2

2 Survey of the State of the Art **3**

2.1 Road Infrastructure Monitoring 3
2.2 Traditional Road Damage Detection Methods 4
2.3 Deep Learning for Road Damage Detection 5
2.4 Evolution of the YOLO Family 7
2.5 Road Damage Datasets 9
2.6 Recent Research in Road Damage Detection 10
2.7 Research Gap 11

3 Methodological Contribution **13**

4 Results and Discussion **14**

5 Conclusion and Perspectives **18**

A Appendix **22**

A.1 Dataset Class Distribution 22
A.2 Additional Confusion Matrices 22
A.3 Precision–Recall Curves 23
A.4 Model Comparison Charts 24
A.5 Cross-Country Generalization Charts 25
A.6 Qualitative Detection Example 27

Introduction

1.1 Background and Motivation

Road transportation plays a vital role in economic development and public safety. However, road pavements gradually deteriorate because of traffic loads, weather conditions, and ageing of construction materials. Common types of pavement distress include longitudinal cracks, transverse cracks, alligator cracks, potholes, and other surface defects. Early detection of these damages is essential to reduce maintenance costs and improve road safety.

Road inspection is traditionally performed through manual visual assessment, which is labour intensive, time consuming, and often subjective. Recent advances in computer vision and deep learning have enabled automatic road damage detection from images. Among modern object detection frameworks, the YOLO family has become one of the most widely used because it provides a good balance between detection accuracy and inference speed, making it suitable for real-time road inspection applications.

1.2 Project Context

This project was carried out as part of the Master of Machine Learning and Data Mining (MLDM) programme at Université Jean Monnet Saint-Étienne. It investigates the use of YOLO-based object detection models for automatic road damage detection using the RDD2022 dataset.

Three models, namely YOLOv5s, YOLOv8n, and YOLOv8s, are trained and evaluated under identical experimental settings. In addition to comparing their detection performance and computational efficiency, the project investigates cross-country generalization by training a model using Japanese road images and evaluating it on road images from other countries. A Streamlit application is also developed to demonstrate the practical deployment of the trained detector.

1.3 Problem Statement

Although many deep learning models have been proposed for road damage detection, direct comparison between published studies remains difficult because different datasets, training procedures, and evaluation protocols are often used. Furthermore, many studies evaluate their models only on images collected under similar conditions, providing limited insight into their ability to generalize to different road environments.

Another practical challenge is achieving a balance between detection accuracy and computational efficiency. Therefore, a fair comparison of modern YOLO models under identical experimental conditions is needed to identify models that are both accurate and suitable for real-time deployment.

1.4 Objectives

The objectives of this project are:

- Train YOLOv5s, YOLOv8n, and YOLOv8s using the RDD2022 dataset.

- Compare the models using Precision, Recall, mAP@0.5, mAP@0.5:0.95, inference time, and FPS.
- Analyse the detection performance of each road damage category.
- Evaluate cross-country generalization by training on Japanese road images and testing on images from India, the Czech Republic, and Norway.
- Develop a Streamlit application for automatic road damage detection.

1.5 Project Workflow

The overall workflow adopted in this project is presented in Figure 1.1. The study begins with dataset preparation and model training. The trained models are then evaluated and compared using standard object detection metrics. The best-performing model is subsequently used in a cross-country generalization experiment and finally integrated into a Streamlit application for practical demonstration.

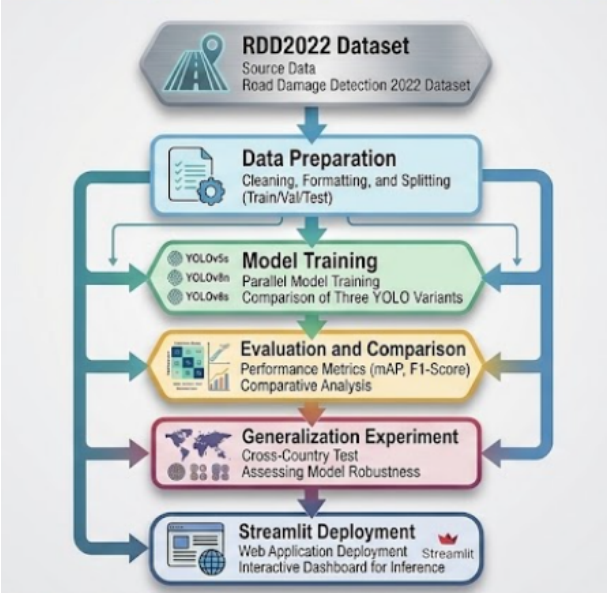


Figure 1.1: Overall workflow of the proposed project.

Survey of the State of the Art

2.1 Road Infrastructure Monitoring

Road transportation plays a vital role in economic development and daily life. Well-maintained roads improve traffic safety, reduce vehicle operating costs, and support the efficient movement of people and goods. However, road pavements gradually deteriorate because of traffic loads, environmental conditions, ageing of construction materials, and insufficient maintenance. Common forms of pavement distress include longitudinal cracks, transverse cracks, alligator cracks, potholes, and other surface defects. If these damages are not detected and repaired at an early stage, they can grow into more serious problems that require costly maintenance and may increase the risk of road accidents.

For many years, road condition assessment has mainly relied on manual visual inspection. In this process, trained inspectors travel along road networks and record the location, type, and severity of pavement damage. Although this approach is widely used, it has several disadvantages. Manual inspection is labour intensive, time consuming, expensive, and often subjective because the final assessment depends on the experience of individual inspectors. In addition, carrying out inspections on busy roads may expose workers to traffic hazards and makes frequent monitoring difficult, especially for large transportation networks. These limitations have encouraged researchers and road authorities to investigate automated approaches for pavement condition monitoring [8, 25].

Recent advances in sensing technologies have made automatic road inspection increasingly practical. Modern road monitoring systems use different types of sensors, including digital cameras, smartphones, laser scanners, LiDAR systems, accelerometers, and unmanned aerial vehicles. Among these technologies, image-based systems have received significant attention because cameras are relatively inexpensive, easy to install, and capable of collecting large amounts of visual information while vehicles travel on normal roads. Images captured by smartphones or vehicle-mounted cameras can then be analysed automatically using computer vision algorithms to identify different types of pavement damage [8].

The increasing availability of large image datasets and improvements in computing hardware have accelerated the adoption of artificial intelligence for road infrastructure monitoring. Instead of relying only on handcrafted image features, modern approaches use machine learning and deep learning techniques to learn useful visual patterns directly from data. These methods have shown strong potential for detecting pavement defects under different lighting conditions, viewing angles, and road textures [25].

Despite these advances, automatic road damage detection remains challenging. Pavement appearance can vary because of construction materials, weather, maintenance practices, road markings, camera position, and image quality. Damage also appears in different shapes and sizes. Thin cracks may occupy only a small part of an image, while potholes and heavily damaged areas can have irregular boundaries. These variations make it difficult for a model trained in one environment to perform equally well in another.

The RDD2022 dataset supports research on this problem by providing road images collected from multiple countries. This enables researchers to study both detection performance and cross-country generalization [2]. This issue is particularly relevant to the present project, which compares several YOLO models and also evaluates how well a model trained on Japanese road images transfers to road images

from other countries.

2.2 Traditional Road Damage Detection Methods

Before the widespread adoption of deep learning, automatic road damage detection mainly relied on traditional image processing and classical machine learning techniques. These approaches usually followed a fixed sequence of operations. First, the image was preprocessed to reduce noise or improve contrast. Next, potential damaged regions were extracted using rules based on intensity, edges, texture, or shape. Finally, the detected regions were classified using either manually defined rules or a conventional machine learning algorithm. Reviews of pavement crack detection commonly group these methods into threshold-based, edge-based, feature-based, and data-driven approaches [11, 28].

One of the simplest approaches is image thresholding. Pavement cracks are often darker than the surrounding road surface, so an image can be converted to grayscale and separated into crack and non-crack regions using an intensity threshold. Global thresholding applies a single threshold to the entire image, while adaptive methods estimate thresholds locally. Otsu's method is also frequently used because it automatically determines a threshold based on the image intensity distribution. Threshold-based methods are computationally efficient and easy to implement, which made them attractive in early pavement inspection systems [11, 28].

However, thresholding depends heavily on the assumption that damaged and undamaged regions have clearly different intensity values. This assumption is often violated in real road images. Shadows, oil stains, road markings, repaired surfaces, water, and variations in illumination can create dark regions that resemble cracks. Similarly, very thin cracks may have weak contrast and can be missed during thresholding. For this reason, threshold-based approaches often require additional preprocessing and post-processing operations before acceptable results can be obtained [4, 19].

Edge detection is another common traditional technique. Crack boundaries produce changes in image intensity, which can be detected using operators such as Sobel, Roberts, Prewitt, or Canny. Among these methods, the Canny detector has been widely used because it combines noise reduction, gradient estimation, non-maximum suppression, and thresholding. Edge-based approaches can identify thin structures effectively when crack boundaries are clearly visible. Nevertheless, pavement images contain many other strong edges, including road markings, aggregate boundaries, joints, shadows, and debris. As a result, an edge detector may identify many structures that are unrelated to actual pavement damage [11, 19].

Morphological image processing has also been used to improve crack extraction. Operations such as erosion, dilation, opening, and closing can remove small noisy regions, connect interrupted crack segments, or refine detected shapes. Filtering techniques, including Gaussian filters, median filters, and Gabor filters, have also been applied to enhance crack patterns before segmentation. These techniques are useful when the visual characteristics of the pavement are relatively stable, but their performance can decrease when images contain complex textures or large variations in lighting conditions [14, 28].

Another group of traditional methods relies on handcrafted visual features. Instead of directly classifying image pixels, these approaches describe the appearance of a road region using manually designed descriptors. Common examples include the Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), Gabor filters, wavelet transforms, and texture statistics. Such descriptors can represent local edge orientation, texture patterns, and frequency information that may help distinguish

damaged pavement from normal pavement [13, 24].

Handcrafted features were often combined with classical machine learning classifiers. Support Vector Machines (SVM), Random Forests, Artificial Neural Networks, k-Nearest Neighbours, and clustering methods have all been applied to infrastructure damage detection. In these systems, the feature extraction stage and the classification stage are separate. The designer first decides which visual features should represent the damage, and the classifier then learns how to separate different classes using those features [8, 25].

Support Vector Machines became particularly popular because they can perform well with relatively small datasets and high-dimensional feature representations. They have been used to distinguish crack and non-crack regions after extracting texture or gradient features. Random Forest models have also been applied because they can combine many decision trees and handle nonlinear relationships between extracted features. These methods provided an important step beyond purely rule-based image processing because the classification decision was learned from labelled examples rather than being completely specified by manually selected thresholds [8, 18].

Despite these improvements, traditional machine learning approaches still depend strongly on the quality of the handcrafted features. A feature representation that works well on one type of pavement may perform poorly when road material, camera resolution, illumination, or damage appearance changes. The feature extraction process also requires domain knowledge and repeated manual tuning. Furthermore, conventional methods have difficulty representing complex visual patterns when several types of damage appear together in realistic road scenes [14, 18].

Another important limitation is that many traditional methods were designed mainly for crack segmentation rather than general road damage detection. Crack extraction focuses on identifying thin linear structures, while real road inspection may also require the detection of potholes, repaired regions, surface deterioration, and multiple damage categories in the same image. A practical system must therefore identify both the type and location of several kinds of road damage. This requirement is more naturally handled as an object detection problem.

Traditional image processing and machine learning methods remain useful because they are often simple, computationally inexpensive, and easy to interpret. They can still perform well in controlled environments where pavement appearance and imaging conditions remain stable. However, their dependence on manually chosen features, thresholds, and processing rules limits their robustness in complex road environments. These limitations encouraged the development of data-driven deep learning approaches, where useful image features can be learned automatically from large collections of labelled examples. The next section discusses this transition and the use of deep neural networks for road damage detection.

2.3 Deep Learning for Road Damage Detection

The rapid development of deep learning has significantly changed the field of computer vision, including automatic road damage detection. Unlike traditional image processing techniques that depend on manually designed features, deep learning models learn hierarchical feature representations directly from training data. This ability allows them to recognize complex visual patterns without requiring handcrafted feature extraction. As larger annotated datasets and more powerful graphics processing units became available, deep learning models began to outperform conventional machine learning methods in many image analysis

tasks, including image classification, object detection, and semantic segmentation [10, 25].

Convolutional Neural Networks (CNNs) form the foundation of most modern vision systems. A CNN automatically learns spatial features by applying convolutional filters to input images. Early layers usually learn simple visual patterns such as edges and corners, while deeper layers learn more complex structures such as textures, shapes, and objects. This hierarchical learning process enables CNNs to capture rich visual information that is difficult to describe using manually designed features. As a result, CNN-based methods have become the dominant approach for pavement damage analysis [10, 14].

Deep learning has been applied to several pavement inspection tasks. Image classification models determine whether an image contains road damage, but they cannot identify the exact location of the defect. Semantic segmentation methods assign a class label to every pixel in the image and provide detailed damage boundaries. Although segmentation produces accurate damage masks, it generally requires pixel-level annotations, which are expensive and time consuming to create. Object detection provides a balance between these two approaches by identifying both the location and the category of each damaged region using bounding boxes. For practical road inspection systems, object detection has become one of the most widely adopted solutions because it combines good accuracy with efficient inference [4, 8].

Modern object detection methods are generally divided into two categories: two-stage detectors and one-stage detectors. Two-stage detectors first generate candidate object regions and then classify each region. One-stage detectors perform localization and classification simultaneously in a single network. This difference has an important effect on detection speed and computational complexity.

One of the earliest successful two-stage detectors was Region-based Convolutional Neural Network (R-CNN). This method first generated thousands of region proposals using an external selective search algorithm and then classified each proposal using a convolutional neural network. Although R-CNN achieved promising detection accuracy, it required repeated feature extraction for every proposed region, making inference very slow [6].

Several improvements were later introduced. Fast R-CNN reduced computation by extracting image features only once and performing classification on shared feature maps. Faster R-CNN further improved efficiency by replacing selective search with a Region Proposal Network (RPN), allowing region proposals to be generated inside the neural network itself. These developments significantly increased detection speed while maintaining high detection accuracy. Consequently, Faster R-CNN became one of the most widely used two-stage object detectors and has been successfully applied to road damage detection in many studies [5, 23].

Although two-stage detectors provide excellent detection performance, their computational requirements often limit real-time deployment. Road inspection systems mounted on moving vehicles require rapid image processing so that damage can be detected while travelling. This requirement motivated the development of one-stage object detectors that perform object localization and classification simultaneously.

Single Shot MultiBox Detector (SSD) was one of the first successful one-stage detectors. SSD predicts object classes and bounding boxes directly from multiple feature maps at different resolutions, allowing objects of different sizes to be detected efficiently. RetinaNet later addressed the class imbalance problem commonly found in dense object detection by introducing the focal loss function. This improvement increased detection accuracy for difficult objects while maintaining the efficiency of one-stage detection [15, 16].

Recent studies show that one-stage detectors have become particularly popular for road damage

detection because they provide an effective balance between detection accuracy and inference speed. This balance is especially important for intelligent transportation systems, autonomous vehicles, and mobile inspection platforms where computational resources and response time are critical. Among one-stage detectors, the You Only Look Once (YOLO) family has attracted considerable attention because of its high processing speed, relatively simple architecture, and competitive detection performance across many object detection benchmarks [8, 24].

The success of YOLO has led to the development of several improved versions that achieve progressively better accuracy and efficiency. These improvements have made YOLO one of the most widely adopted object detection frameworks for infrastructure monitoring and road damage detection. Since the present project compares three different YOLO architectures, it is important to understand how the YOLO family has evolved over time. The next section therefore reviews the development of YOLO from its original version to the latest architectures used in modern road damage detection research.

2.4 Evolution of the YOLO Family

The You Only Look Once (YOLO) family has become one of the most influential object detection frameworks in computer vision. Unlike two-stage detectors that first generate region proposals and then classify them, YOLO treats object detection as a single regression problem. A single neural network simultaneously predicts object locations, confidence scores, and class labels directly from the input image. This unified design significantly reduces computational complexity and enables real-time object detection while maintaining competitive detection accuracy. Since its introduction in 2016, the YOLO family has undergone continuous improvements, with each new version introducing architectural changes that improve speed, accuracy, or training efficiency.

YOLOv1 was proposed by Redmon et al. in 2016 as the first detector to treat object detection as a single end-to-end regression problem, dividing the image into a grid where each cell predicted bounding boxes and class probabilities directly. It achieved remarkable processing speed, but struggled with small objects and had lower localization accuracy than contemporary two-stage detectors [22].

YOLOv2, also known as YOLO9000, added batch normalization, anchor boxes, and multi-scale training to improve both accuracy and robustness over YOLOv1. It also demonstrated the ability to detect more than 9000 object categories by jointly training on detection and classification datasets [20].

YOLOv3 introduced the deeper Darknet-53 backbone and multi-scale prediction at three feature map resolutions, which noticeably improved the detection of small and medium-sized objects. Logistic classifiers also replaced the earlier softmax classifier, allowing more flexible multi-label predictions. These changes made YOLOv3 one of the most widely adopted real-time detectors for several years [21].

YOLOv4 focused on improving accuracy without requiring specialized hardware, combining the CSPDarknet53 backbone with training strategies such as Mosaic data augmentation, Spatial Pyramid Pooling, and a Path Aggregation Network. These additions increased both accuracy and inference speed, making YOLOv4 highly competitive for practical object detection tasks [3].

Although YOLOv5 was never released as an official continuation by the original YOLO authors, it became one of the most widely used implementations in both research and industry. Developed by Ultralytics in PyTorch, it introduced automatic anchor generation, flexible model scaling, and mixed precision training, and is offered in several sizes, including YOLOv5n, YOLOv5s, YOLOv5m, YOLOv5l, and YOLOv5x, allowing users to balance computational cost against detection performance [9].

YOLOv6 and YOLOv7 were later introduced to further improve real-time detection. YOLOv6 redesigned the network architecture for industrial deployment and efficient inference, while YOLOv7 introduced trainable Bag-of-Freebies strategies and model re-parameterization, achieving state-of-the-art performance among real-time detectors at the time of its release [12, 27].

YOLOv8, developed by Ultralytics, is the latest major generation and adopts an anchor-free detection head, which simplifies the prediction process and removes the need for manually selected anchor boxes, together with a more efficient backbone and detection head. It also supports multiple vision tasks, including detection, classification, segmentation, and pose estimation, within a single framework, and is offered in sizes ranging from YOLOv8n to YOLOv8x to balance computational requirements against detection accuracy [26].

The continuous evolution of the YOLO family shows a clear trend toward higher accuracy, better computational efficiency, and easier deployment, which has made it one of the most widely adopted frameworks for road damage detection, where both accuracy and inference speed matter for practical use.

The present project focuses on three representative YOLO models: YOLOv5s, YOLOv8n, and YOLOv8s. YOLOv5s represents a lightweight and well-established detector that has been widely used in previous road damage detection studies. YOLOv8n provides the smallest model in the latest YOLO generation and is designed for applications with limited computational resources. YOLOv8s is a larger variant that offers improved detection accuracy while still maintaining efficient inference. Comparing these three models makes it possible to evaluate how architectural improvements between YOLO generations influence both detection performance and computational efficiency when applied to the RDD2022 road damage dataset.

Table 2.1: Evolution of the YOLO family and its main characteristics.

Version	**Year**	**Main Improvements**	**Remarks**
YOLOv1	2016	Introduced single-stage object detection by predicting bounding boxes and class probabilities in one forward pass.	Very fast detector but limited performance for small objects and localization accuracy.
YOLOv2	2017	Added anchor boxes, batch normalization, high-resolution classifier, and multi-scale training.	Improved localization and detection accuracy while maintaining real-time performance.
YOLOv3	2018	Introduced the Darknet-53 backbone and multi-scale prediction using feature pyramids.	Better detection of small objects and higher overall accuracy.
YOLOv4	2020	Combined CSPDarknet53, Mosaic augmentation, PAN, SPP, and several training optimizations.	Achieved an improved balance between detection accuracy and inference speed.
YOLOv5	2021	PyTorch implementation with automatic anchor generation, model scaling, mixed precision training, and simplified deployment.	Widely adopted in both academia and industry because of its ease of use and strong performance.
YOLOv6	2022	Optimized architecture for industrial deployment and efficient inference.	Focused on practical applications requiring high processing speed.
YOLOv7	2023	Introduced trainable Bag-of-Freebies and model re-parameterization strategies.	State-of-the-art real-time detector at the time of publication.
YOLOv8	2023	Anchor-free detection head, improved backbone, unified framework supporting multiple vision tasks.	Provides high accuracy, efficient inference, and simplified training for modern applications.

Table 2.1 summarizes this evolution, showing how each version brought architectural improvements

that increased accuracy, improved efficiency, or simplified deployment.

2.5 Road Damage Datasets

The success of deep learning models depends not only on the network architecture but also on the quality, size, and diversity of the training dataset. Publicly available datasets provide a common benchmark for evaluating different algorithms under similar conditions and have played an important role in the development of automatic road damage detection systems. Among the available datasets, the Road Damage Detection (RDD) series has become the most widely used benchmark for object detection research.

The first large-scale benchmark, RDD2018, was introduced during the Global Road Damage Detection Challenge. The dataset consisted of road images collected in Japan using smartphone cameras mounted on moving vehicles. It included four categories of road damage annotated with bounding boxes and encouraged researchers to apply deep learning techniques to pavement inspection.

To improve the diversity of the benchmark, RDD2020 extended the dataset by including images collected from India and the Czech Republic in addition to Japan. Compared with RDD2018, the new release introduced greater variation in pavement materials, weather conditions, road textures, and lighting environments. This increased the difficulty of the detection task and encouraged the development of more robust object detection models.

The latest version, RDD2022, represents the largest and most comprehensive benchmark in the RDD series. The original dataset contains 47,420 road images collected from six countries: Japan, India, the Czech Republic, Norway, the United States, and China. More than 55,000 road damage instances are annotated in the PASCAL VOC XML format. The official dataset defines four major road damage categories: longitudinal cracks, transverse cracks, alligator cracks, and potholes. The availability of images from multiple countries makes RDD2022 particularly valuable for evaluating the robustness and generalization capability of deep learning models across different road environments [2].

One of the main strengths of RDD2022 is its geographical diversity. Images were acquired using different platforms, including smartphones, motorcycles, vehicles, and drones. Consequently, the dataset contains considerable variation in pavement materials, camera viewpoints, weather conditions, illumination, and road markings. These characteristics make the dataset significantly more challenging than earlier single-country benchmarks and better represent real-world road inspection scenarios.

Although the official RDD2022 dataset is distributed using the PASCAL VOC annotation format, several research groups have converted it into formats that are directly compatible with modern object detection frameworks. In this project, the publicly available Kaggle version of RDD2022 was used. This version is already converted to the YOLO annotation format and provides predefined training, validation, and test splits, allowing direct use with YOLO-based object detection models without additional annotation conversion [1].

The Kaggle version used in this study contains 26,869 training images, 5,758 validation images, and 5,758 test images. Unlike the original benchmark, this version includes five object classes: longitudinal crack, transverse crack, alligator crack, other corruption, and pothole. The additional *other corruption* category allows the detector to identify a wider range of pavement defects beyond the four major damage classes defined in the original benchmark. This version was selected because it is fully prepared for YOLO training and has been widely adopted by the computer vision community for benchmarking road damage detection models.

Another important advantage of RDD2022 is that it enables cross-country evaluation. Since images are collected from multiple countries, researchers can investigate not only the detection accuracy of a model but also its ability to generalize to unseen road environments. This capability is particularly relevant to the present study, where one of the experiments evaluates how well a model trained only on Japanese road images performs when tested on images collected from other countries.

Overall, the combination of a large number of annotated images, multiple countries, diverse acquisition conditions, and ready-to-use YOLO annotations makes the selected RDD2022 dataset an appropriate benchmark for comparing modern object detection models and analysing their generalization capability.

2.6 Recent Research in Road Damage Detection

The release of public datasets such as RDD2018, RDD2020, and RDD2022 gave researchers a common benchmark to test their models on, and this pushed the field forward quickly. Early work mostly asked a simple question: can deep learning beat older image-processing methods at finding road damage? Once the answer turned out to be yes, attention shifted to making detectors more accurate, lighter to run, and fast enough for real-time use on the road. One of the first studies to shape this area was by Maeda et al. [17], who built the RDD2018 dataset from smartphone photos and tested an SSD-based detector on it. Their results showed that a deep learning model could pick out several types of road damage from ordinary driving footage, and just as importantly, gave later researchers a shared dataset to compare their own methods against. The main weakness they reported was trouble detecting thin cracks when lighting or road surface conditions were poor. Arya et al. [2] later introduced RDD2022, which is much bigger and covers six countries instead of one. Their goal was not just to raise accuracy, but to check whether models could still perform well when the road surface, weather, or camera angle changed from country to country. This idea of testing across different countries is central to the present project as well. As bigger datasets became available, researchers began adapting YOLO models specifically for road damage. Guo and Zhang [7] proposed MN-YOLOv5, which swaps YOLOv5's backbone for MobileNetV3 to make the model smaller and faster. Their tests showed this did not hurt accuracy — mAP actually went up by 2.5More recent work has focused on improving YOLOv8. Zeng and Zhong [29] built YOLOv8-PD, adding an attention module and a lighter detection head to the standard YOLOv8n model. On RDD2022, their changes cut the model's parameters and computation to about three-quarters of the original size while still slightly improving accuracy (mAP up by 1.4 percentage points). This is a good example of a common trade-off in this field: making a model smaller usually costs some accuracy, but careful design can avoid that cost. Zhao et al. [30] took a different route with SEA-YOLOv8, adding attention mechanisms to help the model focus on the more informative parts of an image. Tested on RDD2022, it reached 63.2Taken together, these studies share a few patterns and a few gaps. Most of them modify a single YOLO version and test it only on one dataset split, so it is hard to know from their results alone how one YOLO generation really compares to another under the same conditions. Very few papers report both accuracy and speed together, even though a model that is accurate but too slow is not much use for a vehicle-mounted camera. And although RDD2022 was built specifically to support testing across different countries, most published studies still train and test within a single country's data, leaving the cross-country question largely unanswered. These gaps are exactly what motivate the comparison carried out in this project.

2.7 Research Gap

The literature reviewed in the previous sections shows that significant progress has been made in automatic road damage detection. Modern deep learning models, particularly those belonging to the YOLO family, have substantially improved detection accuracy while achieving real-time inference speeds. The availability of benchmark datasets such as RDD2022 has also encouraged researchers to develop increasingly robust object detection models for pavement inspection.

Despite these advances, several research challenges remain. First, many published studies focus on improving a single object detection architecture by introducing new attention mechanisms, feature fusion modules, or lightweight network components. While these modifications often improve performance for a specific model, relatively few studies provide a direct comparison between different generations of YOLO models under identical experimental conditions. Since different studies frequently use different datasets, training strategies, image resolutions, and evaluation metrics, comparing published results remains difficult.

A second limitation is that many studies primarily report detection accuracy while giving less attention to computational efficiency. In practical road monitoring systems, inference speed is an important consideration because images must often be processed in real time on vehicles or edge devices. A detector with slightly higher accuracy may not always be the most suitable choice if it requires significantly greater computational resources. Therefore, evaluating both detection performance and inference speed provides a more realistic assessment of model suitability for practical deployment. This trade-off has been identified as an important challenge in recent surveys on real-time road damage detection. [8]

Another important challenge is model generalization. Many existing studies train and evaluate their models using randomly divided training and testing sets from the same dataset. Although this evaluation protocol measures detection performance under similar data distributions, it does not fully assess how well a model performs when applied to road images collected under different environmental conditions. Variations in pavement materials, weather, camera viewpoints, illumination, and maintenance practices between countries can significantly affect detection performance. Recent reviews of the CRDDC2022 challenge highlight cross-country robustness as an important direction for future research because practical road inspection systems must operate reliably across diverse geographical environments rather than within a single country. [2]

In addition, relatively few studies combine experimental evaluation with practical deployment. Most published work focuses on model development and benchmark performance, while less attention is given to demonstrating how trained models can be integrated into user-friendly applications that support automatic road inspection in real-world scenarios.

Motivated by these observations, this project performs a comparative evaluation of three representative YOLO models, namely YOLOv5s, YOLOv8n, and YOLOv8s, using the same dataset, identical training configuration, and common evaluation protocol. The study compares detection accuracy, inference speed, and per-class performance to provide a fair assessment of the strengths and limitations of each model. Furthermore, a cross-country generalization experiment is conducted by training a model using only Japanese road images and evaluating it on road images collected from other countries. Finally, the trained detector is integrated into a Streamlit-based application to demonstrate its practical deployment for automatic road damage detection.

The literature review therefore establishes the motivation for the methodological contribution presented

in the next chapter. By combining model comparison, cross-country evaluation, and application deployment within a unified experimental framework, this study addresses several limitations identified in previous research while providing a comprehensive evaluation of modern YOLO architectures for road damage detection.

Methodological Contribution

The experiments were carried out using the publicly available RDD2022 dataset in its YOLO-formatted version available on Kaggle. The dataset contains 38,385 road images divided into predefined training, validation, and test subsets consisting of 26,869, 5,758, and 5,758 images, respectively. Images were collected from six different countries and annotated for five categories of road damage: longitudinal crack, transverse crack, alligator crack, other corruption, and pothole. Before training, the dataset was verified to ensure that every image had a corresponding annotation file and to identify invalid or duplicate labels that could affect the training process.

Three representative YOLO models were selected for comparison: YOLOv5s, YOLOv8n, and YOLOv8s. These models were chosen because they represent different generations of the YOLO family while offering different levels of computational complexity suitable for real-time applications. To ensure a fair comparison, all models were trained using identical experimental settings with an input image size of 640×640 pixels, a batch size of 16, and 50 training epochs. Pre-trained weights supplied by the original implementations were used to initialize the models, allowing transfer learning from large-scale image datasets. A patience value of 15 epochs was adopted for early stopping to avoid unnecessary training when no further improvement was observed.

Training and evaluation were performed using Kaggle Notebooks with GPU acceleration on an NVIDIA Tesla T4 GPU. The Ultralytics implementation was used for YOLOv8n and YOLOv8s, while the official YOLOv5 repository was used for YOLOv5s. Apart from the model architecture, the core training settings, namely image size, batch size, epoch count, and early-stopping patience, were kept identical throughout the study; only minor framework-specific differences in data-loading workers, caching strategy, and optimizer selection were unavoidable between the YOLOv5 and Ultralytics implementations, so the comparison still primarily reflects differences between the models rather than the training configuration.

After training, each model was evaluated on the unseen test set using Precision, Recall, mAP@0.5, mAP@0.5:0.95, and inference speed. In addition to the overall evaluation, the performance of each model was analysed separately for the five road damage categories. This allowed both the detection accuracy and computational efficiency of the models to be assessed under identical testing conditions.

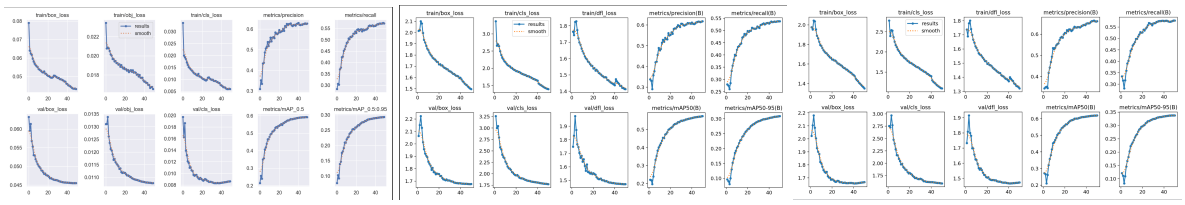
To investigate model robustness, an additional cross-country generalization experiment was conducted using the best-performing model obtained from the initial comparison. Instead of training on images from all six countries, the model was trained exclusively using Japanese road images, which represent the largest subset of the dataset. The trained detector was then evaluated on road images collected from Japan, India, the Czech Republic, Norway, China, and the United States. This experiment was designed to evaluate the ability of the model to generalize to road environments that were not observed during training.

The practical applicability of the proposed approach was demonstrated through the development of a lightweight Streamlit application. The application allows users to upload a road image, performs inference using the trained model, and displays the detected road damages together with their predicted classes and confidence scores. This provides a simple interface for demonstrating the deployment of the trained detector without requiring additional modifications to the model.

Results and Discussion

The proposed methodology was evaluated through three experiments. First, the training behaviour of YOLOv5s, YOLOv8n, and YOLOv8s was analysed to verify that all models converged successfully. The trained models were then evaluated on the unseen RDD2022 test set using standard object detection metrics. Finally, the best-performing model was further investigated through a cross-country generalization experiment. The results obtained from these experiments are presented and discussed in the following sections.

Figure 4.1 presents the training curves of the three models. In all cases, the training and validation losses decreased steadily while Precision, Recall, mAP@0.5 and mAP@0.5:0.95 improved throughout the training process before gradually converging. This behaviour indicates that the selected hyperparameters were appropriate and that all models learned stable feature representations without signs of unstable optimization. Among the three models, YOLOv8s achieved the highest validation performance, whereas YOLOv8n converged faster due to its smaller network architecture.



(a) YOLOv5s (b) YOLOv8n (c) YOLOv8s

Figure 4.1: Training curves of the three YOLO models. Each subfigure presents the variation of training and validation losses together with Precision, Recall, mAP@0.5 and mAP@0.5:0.95 over the 50 training epochs.

The final evaluation was performed using the unseen RDD2022 test set. Table 4.1 summarizes the overall detection performance of the three models. YOLOv8s achieved the highest detection accuracy with a Precision of 0.6598, Recall of 0.5735, mAP@0.5 of 0.6170 and mAP@0.5:0.95 of 0.3329. YOLOv5s obtained intermediate performance with an mAP@0.5 of 0.5860, while YOLOv8n achieved an mAP@0.5 of 0.5712. Although YOLOv8n produced slightly lower detection accuracy, it required only 3.448 ms per image for inference, corresponding to approximately 290 FPS, making it the fastest detector evaluated in this study.

Table 4.1: Overall performance comparison on the RDD2022 test set.

Model	Params (M)	GFLOPs	Precision	Recall	mAP@0.5	mAP@0.5:0.95	FPS
YOLOv5s	7.0	15.8	0.6310	0.5540	0.5860	0.2930	149.25
YOLOv8n	3.0	8.1	0.6181	0.5439	0.5712	0.3056	290.03
YOLOv8s	11.1	28.4	0.6598	0.5735	0.6170	0.3329	104.01

Inference time per image: 6.700 ms (YOLOv5s), 3.448 ms (YOLOv8n), 9.615 ms (YOLOv8s).

The results demonstrate a clear trade-off between detection accuracy and inference speed. YOLOv8s consistently achieved the highest performance across all evaluation metrics, indicating its superior ability

to learn discriminative features for road damage detection. In contrast, YOLOv8n provided the fastest inference speed while maintaining competitive accuracy, making it attractive for real-time applications where computational resources are limited. YOLOv5s provided a balanced compromise between these two models but did not outperform either YOLOv8 architecture. This trade-off is also reflected in model size, since YOLOv8n contains only 3.0 million parameters and requires 8.1 GFLOPs per image, compared with 7.0 million parameters and 15.8 GFLOPs for YOLOv5s, and 11.1 million parameters and 28.4 GFLOPs for YOLOv8s, confirming that the accuracy gains of YOLOv8s come at a proportionally higher computational cost.

To further analyse model behaviour, the detection performance of each road damage category was compared using the mAP@0.5:0.95 metric, as presented in Table 4.2. The results show that "Other Corruption" was the easiest category to detect for all three models, while potholes and transverse cracks remained the most challenging classes. YOLOv8s achieved the highest mAP@0.5:0.95 for every damage category, demonstrating consistent improvements over both YOLOv5s and YOLOv8n. Pothole, which achieved the lowest accuracy for two of the three models, was also the least frequent damage category in the training data, with only 6,544 annotated instances compared to more than 10,000 instances for each of the other classes; this class imbalance likely contributes to its comparatively weaker detection performance.

Table 4.2: Per-class comparison using mAP@0.5:0.95 on the RDD2022 test set.

Damage Class	**YOLOv5s**	**YOLOv8n**	**YOLOv8s**
Longitudinal Crack	0.2890	0.2962	0.3229
Transverse Crack	0.2150	0.2537	0.2782
Alligator Crack	0.3090	0.3387	0.3546
Other Corruption	0.4350	0.4481	0.4730
Pothole	0.2150	0.1913	0.2359

Figure 4.2 shows the normalized confusion matrix obtained using YOLOv8s, which achieved the best overall performance. Most damage categories were correctly classified, with only limited confusion between visually similar crack types. The highest classification accuracy was observed for the "Other Corruption" category, whereas transverse cracks and potholes produced comparatively higher misclassification rates. Overall, the confusion matrix confirms the effectiveness of YOLOv8s for multi-class road damage detection on the RDD2022 benchmark.

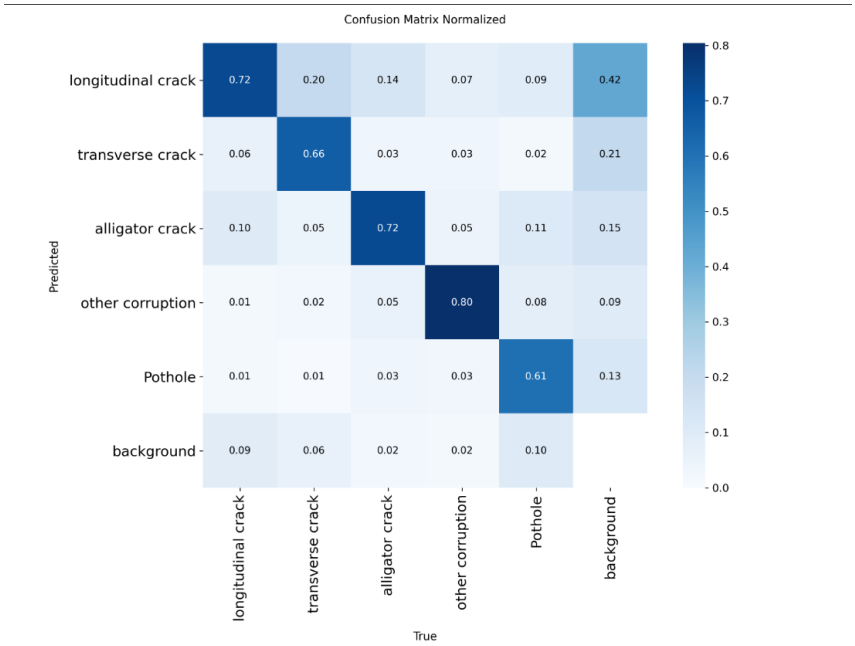


Figure 4.2: Normalized confusion matrix obtained by YOLOv8s on the RDD2022 test set.

The robustness of the best-performing model was further investigated through a cross-country generalization experiment. Unlike the previous experiments, the model was trained exclusively using Japanese road images and evaluated on test images collected from six different countries. This experiment was designed to assess the ability of the detector to generalize to road environments that were not observed during training. The evaluation results are summarized in Table 4.3.

Table 4.3: Cross-country generalization results obtained using the YOLOv8s model trained only on Japanese road images.

Country	**Precision**	**Recall**	**mAP@0.5**	**mAP@0.5:0.95**
Japan	**0.6012**	**0.5883**	**0.5966**	**0.2965**
India	0.1557	0.1593	0.0711	0.0315
Czech Republic	0.2208	0.1840	0.1122	0.0363
Norway	0.2097	0.1129	0.0673	0.0244
China	0.2656	0.2161	0.1581	0.0576
United States	0.4282	0.4927	0.4282	0.2327

As expected, the highest performance was achieved on the Japanese test set because the model was trained exclusively using images from Japan. The detector obtained an mAP@0.5 of 0.5966 and an mAP@0.5:0.95 of 0.2965, demonstrating its ability to accurately detect road damage within the training domain.

When evaluated on road images collected from other countries, the detection performance decreased noticeably. Among the unseen countries, the United States achieved the best generalization performance with an mAP@0.5 of 0.4282 and an mAP@0.5:0.95 of 0.2327. China produced moderate results, whereas India, the Czech Republic and Norway showed considerably lower detection accuracy. The lowest

performance was observed on the Norwegian test set, where the model achieved an mAP@0.5:0.95 of only 0.0244. It should also be noted that the Czech, Norwegian, and United States test subsets do not contain any annotated instances of the Other Corruption category, unlike the Japanese and Indian subsets; part of the performance gap reported for these three countries therefore also reflects differences in class composition between test sets, and not purely a domain shift in visual appearance.

The decrease in performance across unseen countries indicates that geographical differences have a significant impact on model generalization. Variations in pavement materials, road textures, lighting conditions, weather, camera viewpoints and damage characteristics introduce a domain shift between the training and testing data. Although the detector successfully learned the characteristics of Japanese roads, these learned representations were not fully transferable to several other road environments.

Overall, the experimental results demonstrate that YOLOv8s provides the highest detection accuracy among the three evaluated models, while YOLOv8n offers the fastest inference speed. The comparison also highlights the trade-off between accuracy and computational efficiency, where larger models generally achieve better detection performance at the cost of increased inference time. Finally, the cross-country experiment confirms that evaluating a model using randomly divided data from the same dataset is not sufficient to assess its real-world robustness. A detector capable of performing well across different geographical regions requires more diverse training data or domain adaptation techniques to reduce the effect of domain shift.

To demonstrate practical deployment, the best-performing YOLOv8s model was integrated into an interactive Streamlit application. Figure 4.3 shows the application interface after a road image has been uploaded and processed, displaying the original image alongside the annotated detection result together with the corresponding detection table. As shown in the same figure, the sidebar allows the user to adjust the confidence threshold, the IoU threshold, and the inference image size, giving direct control over detection sensitivity without requiring changes to the underlying model.

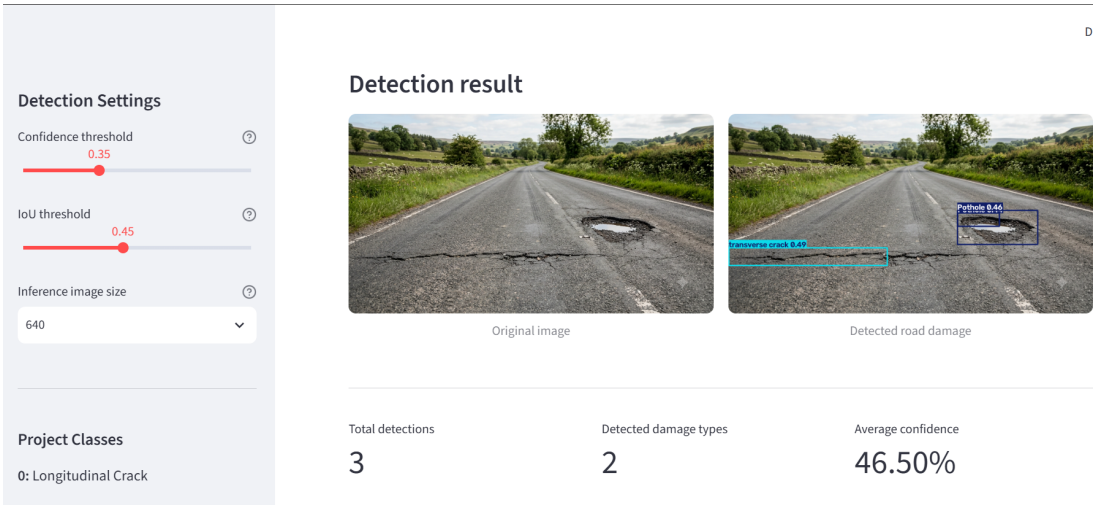


Figure 4.3: Streamlit application showing an uploaded road image, the annotated detection result, and the corresponding detection table.

Conclusion and Perspectives

This project investigated the application of deep learning for automatic road damage detection using the RDD2022 dataset. Three state-of-the-art object detection models, namely YOLOv5s, YOLOv8n, and YOLOv8s, were trained and evaluated under identical experimental settings to provide a fair comparison of their detection accuracy and computational efficiency. In addition to the standard evaluation on the RDD2022 test set, a cross-country generalization experiment was conducted to assess the robustness of the best-performing model when applied to road images collected from different countries. A Streamlit application was also developed to demonstrate the practical deployment of the trained model.

The experimental results showed that YOLOv8s achieved the highest overall detection performance, obtaining the best Precision, Recall, mAP@0.5, and mAP@0.5:0.95 among the three evaluated models. In contrast, YOLOv8n provided the fastest inference speed, making it a suitable choice for applications where computational efficiency is a priority. The per-class analysis further showed that the *Other Corruption* category was detected with the highest accuracy, whereas potholes and transverse cracks remained the most challenging damage types for all models.

The cross-country generalization experiment highlighted an important limitation of the trained detector. Although the model performed well on Japanese road images, its detection accuracy decreased considerably when evaluated on road images from other countries. This indicates that differences in road surfaces, environmental conditions, and image characteristics introduce a significant domain shift that affects model performance. These findings demonstrate that high accuracy on a benchmark dataset does not necessarily guarantee robust performance in different geographical environments.

Overall, the results indicate that YOLOv8s provides the best balance between detection performance and practical applicability for the RDD2022 dataset. The developed Streamlit application further demonstrates that the trained detector can be integrated into a simple and interactive system for automatic road damage detection.

Future work may focus on improving cross-country generalization by incorporating more geographically diverse training data or applying domain adaptation techniques. Additional improvements could also be achieved by investigating recent YOLO variants, advanced data augmentation strategies, and model optimization techniques for deployment on resource-constrained edge devices.

Bibliography

[1] Ali Abdelmenam. Rdd 2022: Road damage detection dataset. `https://www.kaggle.com/datasets/aliabdelmenam/rdd-2022`, 2024. Kaggle dataset, accessed 2026.

[2] Deeksha Arya, Hiroya Maeda, Sanjay Kumar Ghosh, Durga Toshniwal, and Yoshihide Sekimoto. Rdd2022: A multi-national image dataset for automatic road damage detection. *arXiv preprint arXiv:2209.08538*, 2022.

[3] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[4] Zhenyu Du, Jie Yuan, Feipeng Xiao, and Chamod Hettiarachchi. Application of image technology on pavement distress detection: A review. *Measurement*, 184:109900, 2021. doi: 10.1016/j.measurement.2021.109900.

[5] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1440–1448, 2015.

[6] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 580–587, 2014.

[7] Gege Guo and Zhenyu Zhang. Road damage detection algorithm for improved yolov5. *Scientific Reports*, 12:15523, 2022. doi: 10.1038/s41598-022-19674-8.

[8] Muhammad Saiful Islam et al. Advancement in the automation of paved roadways performance patrolling: A review. *Measurement*, 232:114734, 2024.

[9] Glenn Jocher et al. Yolov5. https://github.com/ultralytics/yolov5, 2021.

[10] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[11] Bin Li et al. A critical review and comparative study on image segmentation-based techniques for pavement crack detection. *Construction and Building Materials*, 321:126162, 2022. doi: 10.1016/j.conbuildmat.2021.126162.

[12] Chuyi Li et al. Yolov6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.

[13] Hui Li et al. Automatic pixel-level pavement crack recognition using a deep feature aggregation segmentation network with a scse attention mechanism module. *Sensors*, 21(9):2902, 2021. doi: 10.3390/s21092902.

[14] Y. Li et al. A state-of-the-art survey of deep learning models for automated pavement crack segmentation. *International Journal of Transportation Science and Technology*, 2023.

[15] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.

[16] Wei Liu, Dragomir Anguelov, Dumitru Erhan, et al. Ssd: Single shot multibox detector. In *European Conference on Computer Vision*, pages 21–37, 2016.

[17] Hiroya Maeda, Yoshihide Sekimoto, Toshikazu Seto, Takehiro Kashiyama, and Hiroshi Omata. Road damage detection and classification using deep neural networks with smartphone images. *Computer-Aided Civil and Infrastructure Engineering*, 33(12):1127–1141, 2018. doi: 10.1111/mice.12387.

[18] Farid Mohammad et al. Crack detection in concrete structures using deep learning. *Sustainability*, 14(13):8117, 2022.

[19] Anju Mohan and Sumathi Poobal. Crack detection using image processing: A critical review and analysis. *Alexandria Engineering Journal*, 57(2):787–798, 2018. doi: 10.1016/j.aej.2017.01.020.

[20] Joseph Redmon and Ali Farhadi. Yolo9000: Better, faster, stronger. In *CVPR*, 2017.

[21] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[22] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016.

[23] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[24] Jing Shang, Allen A. Zhang, Zishuo Dong, Hang Zhang, and Anzheng He. Automated pavement detection and artificial intelligence pavement image data processing technology. *Automation in Construction*, 2024.

[25] Nima Sholevar, Amir Golroo, and Sahand Roghani Esfahani. Machine learning techniques for pavement condition evaluation. *Automation in Construction*, 136:104190, 2022. doi: 10.1016/j.autcon.2022.104190.

[26] Ultralytics. Ultralytics yolov8 documentation. https://docs.ultralytics.com, 2023.

[27] Chien-Yao Wang et al. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. *CVPR*, 2023.

[28] Weixing Wang et al. Pavement crack image acquisition methods and crack extraction algorithms: A review. *Journal of Traffic and Transportation Engineering*, 6(6):535–556, 2019. doi: 10.1016/j.jtte.2019.10.001.

[29] Jiayi Zeng and Han Zhong. Yolov8-pd: An improved road damage detection algorithm based on yolov8n model. *Scientific Reports*, 14:12052, 2024. doi: 10.1038/s41598-024-62933-z.

[30] Yuxi Zhao, Baoyong Shi, Xiaoguang Duan, Wenxing Zhu, Liying Ren, and Chang Liao. Research on road surface damage detection based on sea-yolov8. *PLOS ONE*, 2025. doi: 10.1371/journal.pone.0324439.

Appendix

This appendix provides supplementary figures that support the discussion presented in Chapters 3 and 4, including the class distribution of the dataset, additional confusion matrices and precision–recall curves for the models not shown in the main text, the detailed model-comparison charts, the cross-country generalization charts, and a qualitative detection example.

A.1 Dataset Class Distribution

Figure A.1 shows the number of annotated instances for each of the five road damage categories across the combined training, validation, and test sets. Longitudinal cracks are by far the most frequent category, while potholes are the least represented, which is consistent with the comparatively lower detection accuracy obtained for potholes in Chapter 4.

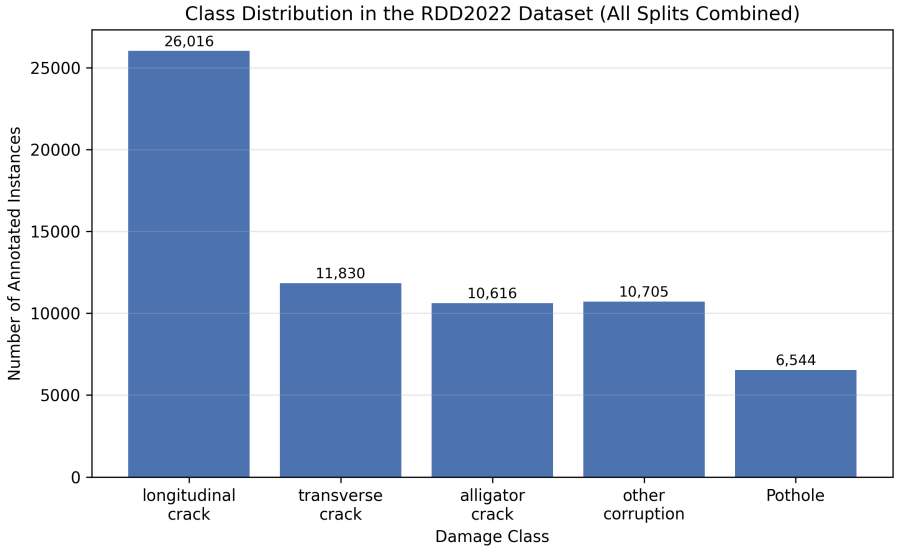


Figure A.1: Class distribution in the RDD2022 dataset (all splits combined).

A.2 Additional Confusion Matrices

Figures A.2 and A.3 present the confusion matrices obtained for YOLOv5s and YOLOv8n on the RDD2022 test set, complementing the YOLOv8s confusion matrix shown in Figure 4.2.

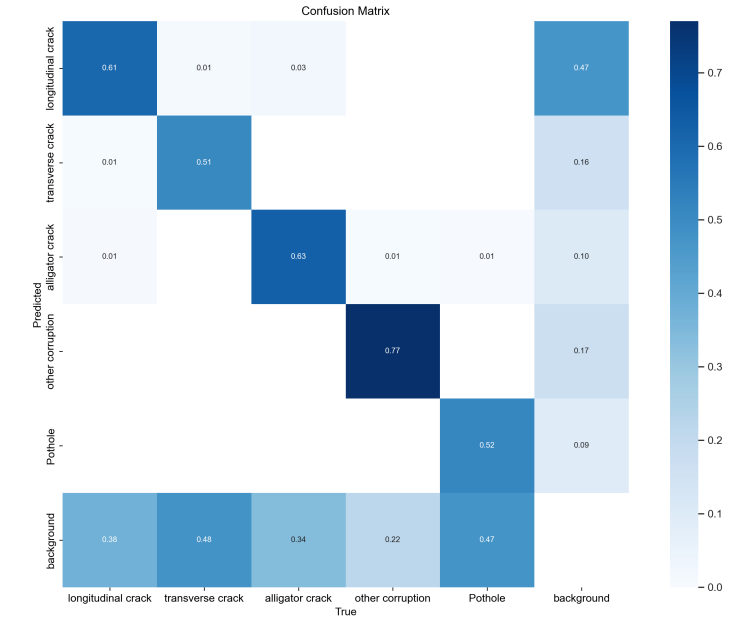


Figure A.2: Confusion matrix (raw counts) obtained by YOLOv5s on the RDD2022 test set.

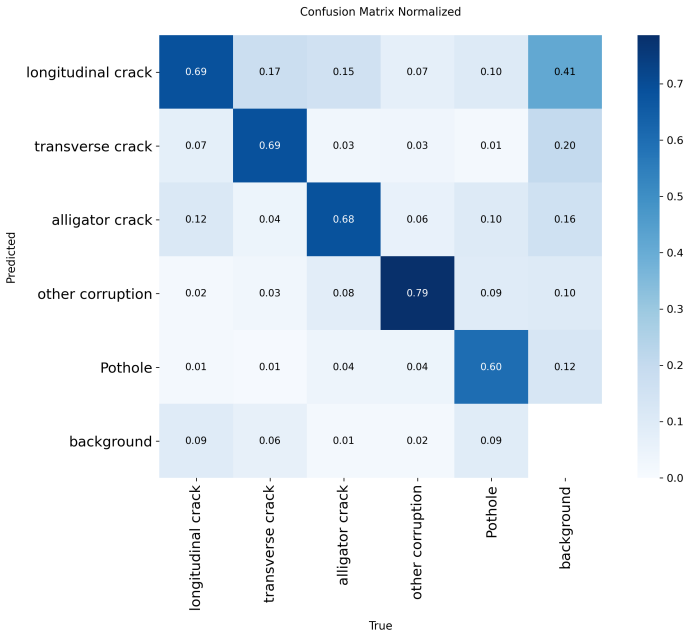
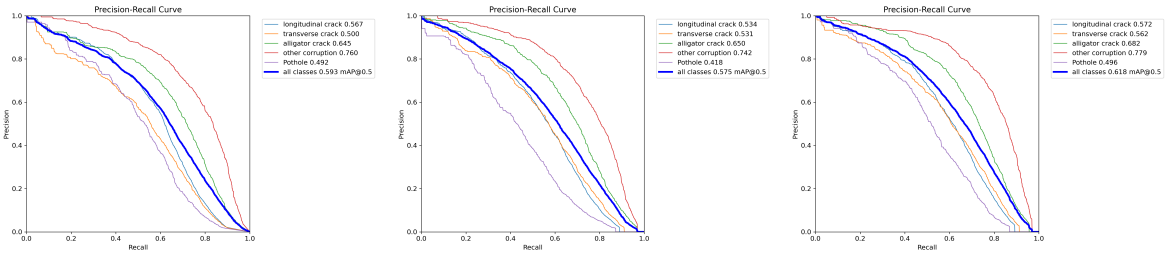


Figure A.3: Normalized confusion matrix obtained by YOLOv8n on the RDD2022 test set.

A.3 Precision–Recall Curves

Figure A.4 presents the precision–recall curves obtained for the three models on the RDD2022 test set.



(a) YOLOv5s (b) YOLOv8n (c) YOLOv8s

Figure A.4: Precision–recall curves obtained by the three YOLO models on the RDD2022 test set.

A.4 Model Comparison Charts

Figures A.5–A.8 present the model comparison results of Table 4.1 and Table 4.2 in chart form.

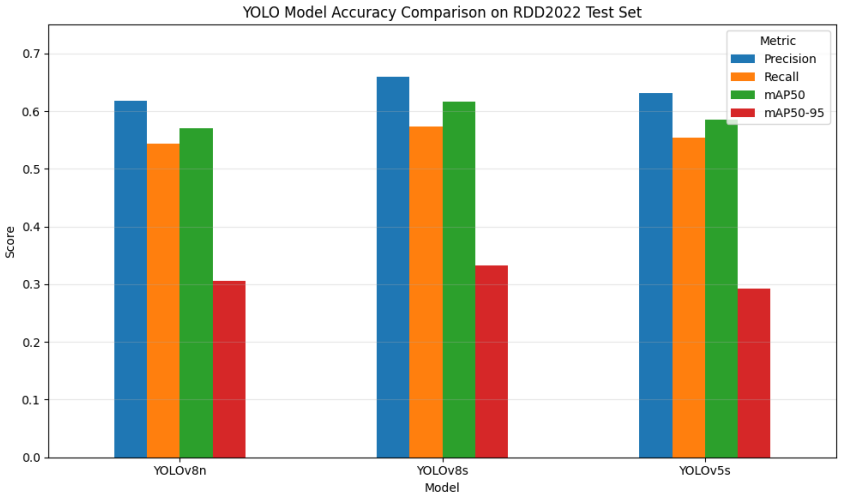


Figure A.5: Accuracy comparison of the three YOLO models on the RDD2022 test set.

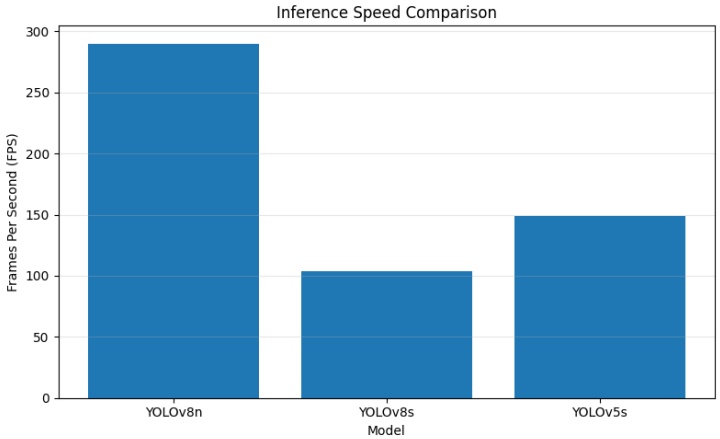


Figure A.6: Inference speed (FPS) comparison of the three YOLO models.

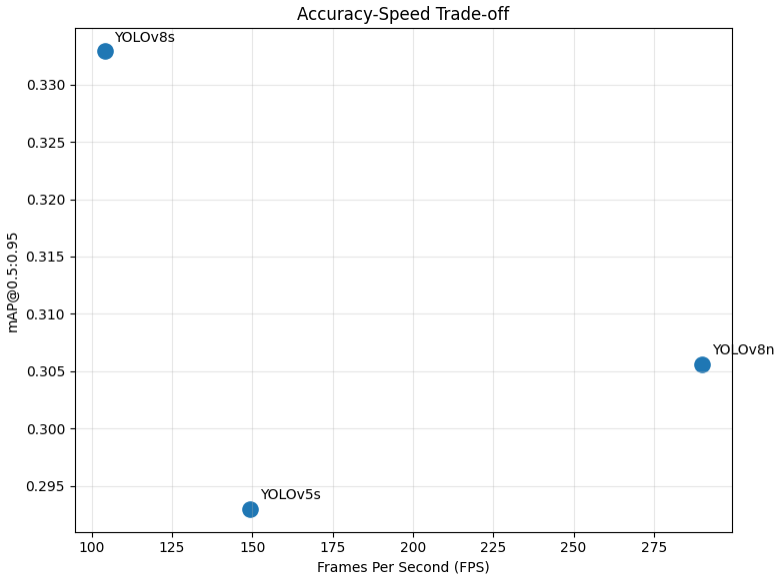


Figure A.7: Accuracy–speed trade-off of the three YOLO models.

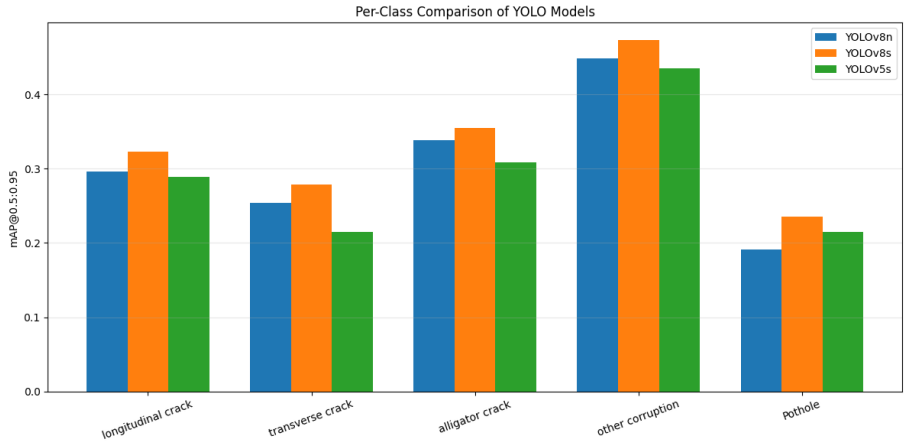


Figure A.8: Per-class mAP@0.5:0.95 comparison of the three YOLO models.

A.5 Cross-Country Generalization Charts

Figures A.9–A.12 provide additional detail on the cross-country generalization experiment discussed in Chapter 4.

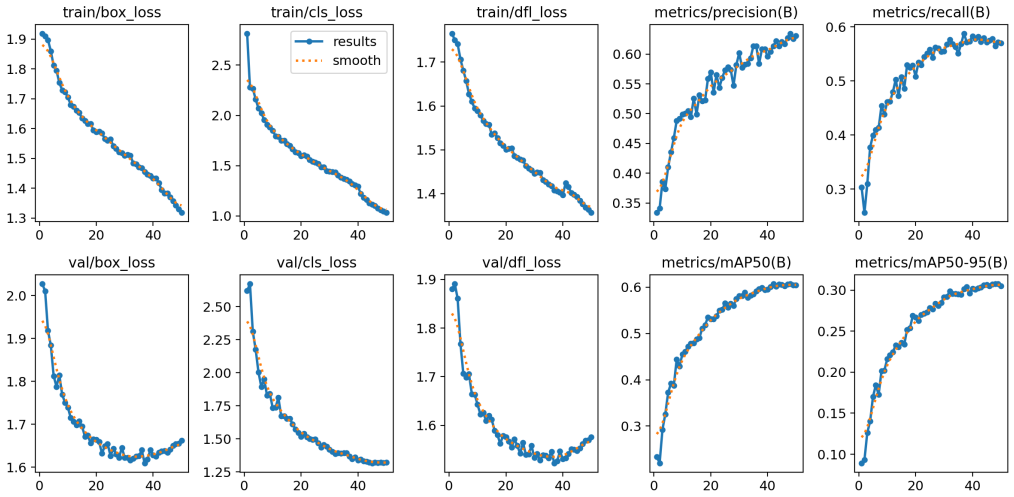


Figure A.9: Training curves of YOLOv8s trained exclusively on Japanese road images.

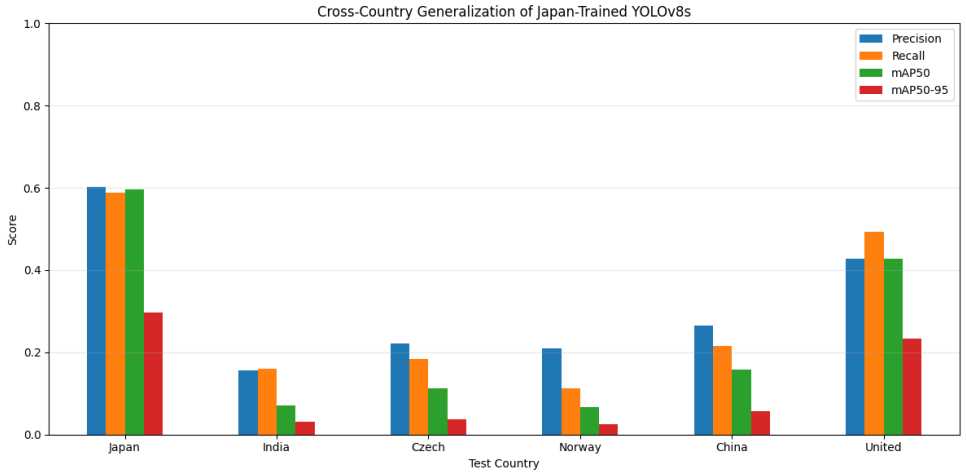


Figure A.10: Precision, Recall, mAP@0.5 and mAP@0.5:0.95 of the Japan-trained YOLOv8s model across all six countries.

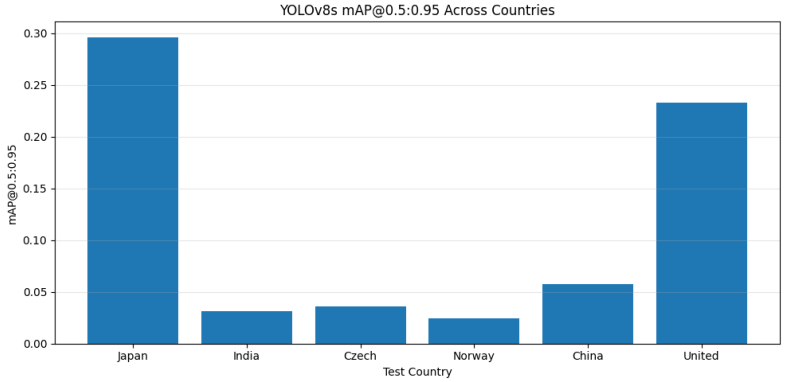


Figure A.11: mAP@0.5:0.95 of the Japan-trained YOLOv8s model by country.

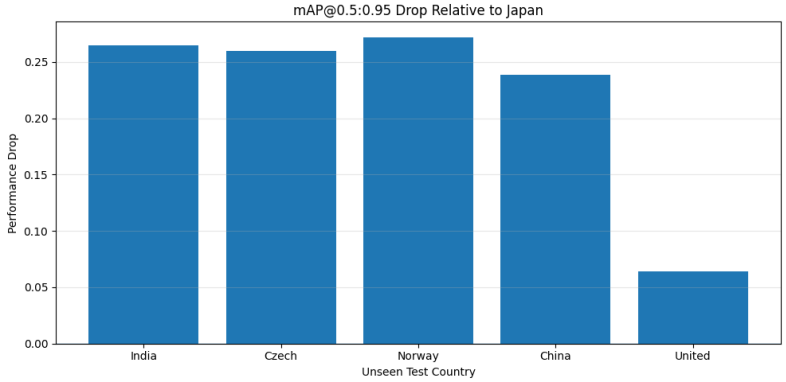


Figure A.12: mAP@0.5:0.95 drop relative to Japan for each unseen test country.

A.6 Qualitative Detection Example

Figure A.13 shows an example of road damage detections produced by the YOLOv8s model on a batch of validation images, illustrating predicted bounding boxes, class labels, and confidence scores.



Figure A.13: Example of qualitative detection results produced by YOLOv8s.